SD-001 System design revision cheat sheet

Scalability, latency, throughput and availability

Fast for one user does not mean it scales.

The goal: maximum throughput at acceptable latency.

Four ideas, first with a chai stall, then with requests

Latency

Chai: how long one person waits for one cup. Say, 2 minutes.

Your service
Time to serve one request.

Measured in ms. Lower is better.

Throughput

Chai: how many cups the stall serves in an hour.

Your service
Requests served per second.

Measured in requests/sec. Higher is better.

Availability

Chai: is the stall open and actually serving?

Your service
How often it works when someone asks.

Measured as a percentage, in nines.

Scalability

Chai: the crowd grows. Add another counter and still keep up? Then the stall scales.

Your service
Keep performance as the work changes, by adding or removing resources.

Performance vs scalability

The classic interview trap

Quick check. A page loads in 200 ms for one user. Does that prove it scales?
No. That is only the latency for one user.

Performance problem	Slow even for a single user.
Scalability problem	Fast for one user, slow when the load grows.

Say it in the interview: "Fast alone but slow at peak is a scalability problem."

How to add capacity

Vertical (scale up)	Add compute to the existing machine. Suits work that is hard to split or is stateful. Every resource has an upper limit.
Horizontal (scale out)	Add more instances. Suits work that splits into independent parts. Gives resilience.

The trap: the average hides the slow users

The typical request takes 50 ms. Looks great. But could many users still wait about a second? Yes.



● 95 requests at 50 ms ● 5 requests at 1000 ms



Average = (95 × 50 + 5 × 1000) ÷ 100 = 97.5, about 98 ms. It looks fine, but 5 out of every 100 users wait 20× longer.

P50 (the median)
Half of the requests are faster than this. It shows the typical case.

P99
99 out of 100 requests are faster. It shows a realistic worst case.

Also: measure failed requests separately. A fast error is still an error (and a slow error is even worse).

Availability and the nines

Downtime allowed per year (365-day year). Bars are not to scale.

99%	**3.65 days**
99.9%	**8.76 hours**
99.99%	**52.6 minutes**
99.999%	**5.26 minutes**

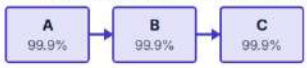
Every extra nine cuts downtime to a tenth, **but costs more**. 100% is simply not achievable.

Two ways to write it
successful requests ÷ all requests
uptime ÷ (uptime + downtime)

Chains and copies

An interview favourite. Each service is 99.9% available.

In a chain: all three must work



0.999 × 0.999 × 0.999 ≈ 99.7%
Lower than any single service. **Multiply.**

In parallel: only one must work



1 − (0.001 × 0.001) = 99.9999%
Copies raise availability. **Chains lower it.**

SLI, SLO, SLA

How teams set targets

SLI	**The measure**	Example: the share of requests that succeed.
SLO	**The target**	Example: 99.9% over four weeks.
SLA	**The contract**	Miss it and there are consequences.

Error budget = the gap left by the SLO.
99.9% over 3 million requests allows **3,000 failures**.

Your 30-second interview answer

Say it in this order

> Scalability is how well a system handles changing load, by adding or removing resources.

> I judge it with latency, using P99 and not the average, and with throughput.

> For availability, I set an SLO on an SLI, and the SLA is the contract.

> In a chain of services, availabilities multiply, so the whole is less available than any single part.

If they ask "why P99 and not the average?" The average hides the slow tail.

Recap

1. Latency is time per request. Throughput is requests per second.
2. Use percentiles, not averages.
3. SLI measures, SLO targets, SLA binds. Chains multiply availability down.

Test yourself

200 ms for one user. Does it scale? **No. It only gives latency for one user.**

Three 99.9% services in a chain? **About 99.7%.**

99.99% allows how much downtime a year? **About 52.6 minutes.**

Which metric exposes the slow 5%? **P99.**

Sources: Google SRE Book and Workbook, AWS Well-Architected Reliability Pillar, Azure Well-Architected, System Design Primer. Downtime figures use a 365-day year.